

Elementary Number Theory

Elementary Number Theory

Author

MUHO CHOI

Email: sihyeon8240@gmail.com

GitHub: <https://github.com/sihyeon8240>

Source Code & Contributions

GitHub: <https://github.com/sihyeon8240/math>

Feel free to open issues or pull requests for corrections.

© 2026. MUHO CHOI.

This textbook is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/) (CC BY 4.0).

You are free to: Share, copy, and redistribute the material in any medium or format; adapt, remix, transform, and build upon the material for any purpose, even commercially.

Under the following terms: You must give appropriate credit, provide a link to the license, and indicate if changes were made.

Contents

Preface	iv
1 Divisibility Theory in the Integers	1
1.1 The Division Algorithm	1
1.2 The Greatest Common Divisor	4
1.3 The Euclidean Algorithm	16
1.4 The Diophantine Equation $ax+by=c$	25
2 Primes and Their Distribution	29
2.1 The Fundamental Theorem of Arithmetic	29
2.2 The Sieve of Eratosthenes	39
2.3 The Goldbach Conjecture	46
3 The Theory of Congruences	53
3.1 Basic Properties of Congruence	53
3.2 Binary and Decimal Representations of Integers	53
3.3 Linear Congruences and the Chinese Remainder Theorem	53
4 Fermat's Theorem	54
4.1 Fermat's Little Theorem and Pseudoprimes	54
4.2 Wilson's Theorem	54
4.3 The Fermat-Kraitchik Factorization Method	54
5 Number-Theoretic Functions	55
5.1 The Sum and Number of Divisors	55
5.2 The Möbius Inversion Formula	55
5.3 The Greatest Integer Function	55
5.4 An Application to the Calendar	55
6 Euler's Generalization of Fermat's Theorem	56
6.1 Euler's Phi-Function	56
6.2 Euler's Theorem	56
6.3 Some Properties of the Phi-Function	56

7	Primitive Roots and Indices	57
7.1	The Order of an Integer Modulo n	57
7.2	Primitive Roots for Primes	57
7.3	Composite Numbers Having Primitive Roots	57
7.4	The Theory of Indices	57
8	The Quadratic Reciprocity Law	58
8.1	Euler's Criterion	58
8.2	The Legendre Symbol and Its Properties	58
8.3	Quadratic Reciprocity	58
8.4	Quadratic Congruences with Composite Moduli	58
9	Introduction to Cryptography	59
9.1	From Caesar Cipher to Public Key Cryptography	59
9.2	The Knapsack Cryptosystem	59
9.3	An Application of Primitive Roots to Cryptography	59
10	Numbers of Special Form	60
10.1	Perfect Numbers	60
10.2	Mersenne Primes and Amicable Numbers	60
10.3	Fermat Numbers	60
11	Certain Nonlinear Diophantine Equations	61
11.1	The Equation $x^2 + y^2 = z^2$	61
11.2	Fermat's Last Theorem	61
12	Representation of Integers as Sums of Squares	62
12.1	Sums of Two Squares	62
12.2	Sums of More Than Two Squares	62
13	Fibonacci Numbers	63
13.1	The Fibonacci Sequence	63
13.2	Certain Identities Involving Fibonacci Numbers	63
14	Continued Fractions	64
14.1	Finite Continued Fractions	64
14.2	Infinite Continued Fractions	64
14.3	Farey Fractions	64
14.4	Pell's Equation	64
15	Some Modern Developments	65
15.1	Primality Testing and Factorization	65
15.2	An Application to Factoring: Remote Coin Flipping	65
15.3	The Prime Number Theorem and Zeta Function	65

Preface

This textbook uses Lean 4 to formalize selected mathematical statements and proofs.

- **Tools.** Lean is an interactive theorem prover; Mathlib is a library of formalized mathematics. This edition uses Lean 4.34.0 and Mathlib v4.34.0 (revision [5ed296525643](#)).
- **Reading the proofs.** Each paired statement or proof step gives a mathematical explanation followed by its Lean code. The explanations can be read independently; code fragments are read in order within the complete declaration.
- **Verification.** Lean checks formal proofs under their stated assumptions. Verification covers only statements linked to checked declarations. Agreement between the explanations, displayed code, and checked proofs is reviewed separately.

Chapter 1

Divisibility Theory in the Integers

1.1 The Division Algorithm

Theorem 1.1.1. *Let $a \in \mathbb{Z}$ and $b \in \mathbb{N}^+$. There is a unique pair $(q, r) \in \mathbb{Z} \times \mathbb{N}$ such that*

$$a = qb + r, \quad r < b.$$

*We call q the **quotient** and r the **remainder**.*

```
theorem existsUnique_quotient_remainder (a : ℤ) (b : ℕ+) :  
  ∃! qr : ℤ × ℕ,  
    a = qr.1 * b + qr.2 ∧ qr.2 < b
```

Proof. Since $b \geq 1$,

$$a - (-|a|)b = a + |a|b \geq a + |a| \geq 0.$$

```
classical  
  
have hb : (1 : ℤ) ≤ (b : ℤ) := by  
  exact_mod_cast b.pos  
  
have hnonneg : 0 ≤ a - (-|a|) * (b : ℤ) := by  
  have ha : -|a| ≤ a := neg_abs_le a  
  have habs : 0 ≤ |a| := abs_nonneg a  
  nlinarith
```

Define

$$S := \{n \in \mathbb{N} : a - qb = n \text{ for some } q \in \mathbb{Z}\}.$$

By the preceding inequality, $S \neq \emptyset$. Let $r = \min S$ and choose $q \in \mathbb{Z}$ with $a - qb = r$.

```

let h :  $\exists n : \mathbb{N}, \exists q : \mathbb{Z}, a - q * b = (n : \mathbb{Z}) :=$ 
  ((a - (-|a|) * b).toNat, -|a|, (Int.toNat_of_nonneg hnonneg).symm)

obtain (q, hq) := Nat.find_spec h

let r :  $\mathbb{N} :=$  Nat.find h

have hr0 : (0 :  $\mathbb{Z}$ )  $\leq$  r := Int.natCast_nonneg _

```

By minimality, for $s \in \mathbb{Z}$,

$$0 \leq s, \quad s = a - xb \ (x \in \mathbb{Z}) \implies r \leq s.$$

```

have hminimal :
   $\forall s : \mathbb{Z}, 0 \leq s \rightarrow (\exists x : \mathbb{Z}, a - x * b = s) \rightarrow (r : \mathbb{Z}) \leq s :=$  by
  intro s hs (x, hx)

have hn : Nat.find h  $\leq$  s.toNat :=
  Nat.find_min' h (x, hx.trans (Int.toNat_of_nonneg hs).symm)

have hc : (Nat.find h :  $\mathbb{Z}$ )  $\leq$  (s.toNat :  $\mathbb{Z}$ ) :=
  Int.ofNat_le.mpr hn

  rwa [Int.toNat_of_nonneg hs] at hc

```

If $r \geq b$, then

$$0 \leq r - b = a - (q + 1)b < r,$$

contradicting minimality. Hence $r < b$.

```

have hrb_int : (r :  $\mathbb{Z}$ ) < b := by
  by_contra hnot

have hs :  $0 \leq (r : \mathbb{Z}) - b :=$  by
  linarith

have hsmaller := hminimal ((r :  $\mathbb{Z}$ ) - b) hs (q + 1, by
  dsimp [r]
  nlinarith [hq])

  linarith

```

Thus $a = qb + r$ and $0 \leq r < b$, proving existence.

```

have ha : a = q * b + r := by
  dsimp [r]
  linarith [hq]

have hrb : r < (b : ℕ) := by
  exact_mod_cast hrb_int

refine ((q, r), (ha, hrb), ?_)

```

Suppose another pair $(q', r') \in \mathbb{Z} \times \mathbb{N}$ satisfies $a = q'b + r'$ and $0 \leq r' < b$.

```

rintro (q', r') (ha', hr'b)

change a = q' * b + r' at ha'

have hr'0 : (0 : ℤ) ≤ r' := Int.natCast_nonneg _

have hr'b_int : (r' : ℤ) < b := by
  exact_mod_cast hr'b

```

Since $q, q' \in \mathbb{Z}$,

$$\begin{aligned}
 q' < q &\implies q - q' \geq 1 \implies r' = r + (q - q')b \geq b, \\
 q' > q &\implies q' - q \geq 1 \implies r = r' + (q' - q)b \geq b.
 \end{aligned}$$

Both contradict the remainder bounds, so $q' = q$.

```

have hqq : q' = q := by
  rcases lt_trichotomy q' q with hlt | heq | hgt

  · have hstep : q' + 1 ≤ q := hlt
    nlinarith

  · exact heq

  · have hstep : q + 1 ≤ q' := hgt
    nlinarith

```

Then $r' = a - q'b = a - qb = r$, so $(q', r') = (q, r)$, proving uniqueness.

```

have hrr_int : (r' : ℤ) = r := by
  rw [hqq] at ha'
  linarith

have hrr : r' = r := by
  exact_mod_cast hrr_int

exact Prod.ext hqq hrr

```

□

1.2 The Greatest Common Divisor

For integers c, z , define $c \mid z$ to mean that $z = ck$ for some $k \in \mathbb{Z}$. In particular, $0 \mid z$ if and only if $z = 0$.

Theorem 1.2.1. *Suppose $a, b \in \mathbb{Z}$ are not both zero. There is a positive natural number d which divides both a and b , is an integer linear combination of them, and is divisible by every common divisor.*

```

theorem exists_positive_common_divisor (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) :
  ∃ d : ℕ, 0 < d ∧ (d : ℤ) ∣ a ∧ (d : ℤ) ∣ b ∧
    (∃ x y : ℤ, (d : ℤ) = a * x + b * y) ∧
    (∀ c : ℤ, c ∣ a → c ∣ b → c ∣ (d : ℤ))

```

Proof. Since a, b are not both zero, $a^2 + b^2 > 0$. Thus the set of positive natural numbers of the form $ax + by$, with $x, y \in \mathbb{Z}$, is nonempty.

```

classical

have hpos : 0 < a * a + b * b := by
  rcases hab with ha | hb

  · nlinarith [sq_pos_of_ne_zero ha, sq_nonneg b]
  · nlinarith [sq_nonneg a, sq_pos_of_ne_zero hb]

have hS : ∃ n : ℕ, 0 < n ∧ ∃ x y : ℤ, a * x + b * y = (n : ℤ) := by
  refine ((a * a + b * b).toNat, _, a, b, _)

  · omega
  · exact (Int.toNat_of_nonneg (le_of_lt hpos)).symm

```

Let d be its least element and choose $x, y \in \mathbb{Z}$ with $ax + by = d$. Every positive natural number represented by such a combination is at least d .

```

let d : ℕ := Nat.find hS

obtain ⟨hdpos, x, y, hxy⟩ := Nat.find_spec hS

have hminimal (n : ℕ) (hn : 0 < n)
  (hlinear : ∃ u v : ℤ, a * u + b * v = (n : ℤ)) : d ≤ n :=
  Nat.find_min' hS ⟨hn, hlinear⟩

```

Suppose $u, v \in \mathbb{Z}$. Apply the division algorithm to $au + bv$ and the positive divisor d , obtaining $au + bv = qd + r$ with $0 \leq r < d$.

```

have hddiv (u v : ℤ) : (d : ℤ) ∣ a * u + b * v := by
  obtain ⟨⟨q, r⟩, ⟨heq, hr⟩, _⟩ :=
    existsUnique_quotient_remainder (a * u + b * v) ⟨d, hdpos⟩

  change a * u + b * v = q * (d : ℤ) + (r : ℤ) at heq
  change r < d at hr

```

The remainder is again a linear combination: $r = a(u - qx) + b(v - qy)$.

```

have hrlinear : a * (u - q * x) + b * (v - q * y) = (r : ℤ) := by
  calc
    a * (u - q * x) + b * (v - q * y) =
      (a * u + b * v) - q * (a * x + b * y) := by
        ring
    _ = r := by
      rw [hxy]
      linarith [heq]

```

If $r \neq 0$, then $r > 0$ and minimality gives $d \leq r$, contradicting $r < d$. Hence $r = 0$.

```

have hrzero : r = 0 := by
  by_contra hne

  have hle := hminimal r (Nat.pos_of_ne_zero hne)
    ⟨u - q * x, v - q * y, hrlinear⟩

  omega

```

Therefore $d \mid au + bv$. Taking $(u, v) = (1, 0)$ and $(0, 1)$ gives $d \mid a$ and $d \mid b$.

```

refine (q, ?_)
rw [hrzero] at heq
simp only [Nat.cast_zero, add_zero, mul_comm] using heq

have hda : (d : ℤ) | a := by
  simp only [mul_one, mul_zero, add_zero] using hddiv 1 0

have hdb : (d : ℤ) | b := by
  simp only [mul_one, mul_zero, zero_add] using hddiv 0 1

```

Suppose $c \mid a$ and $c \mid b$. Choose $u, v \in \mathbb{Z}$ with $a = cu$ and $b = cv$. Then $d = ax + by = c(ux + vy)$, so $c \mid d$. This proves all the asserted properties.

```

have hcommon (c : ℤ) (hca : c ∣ a) (hcb : c ∣ b) : c ∣ (d : ℤ) := by
  obtain ⟨u, hu⟩ := hca
  obtain ⟨v, hv⟩ := hcb

  refine (u * x + v * y, ?_)
  rw [← hxy, hu, hv]
  ring

exact ⟨d, hdpow, hda, hdb, ⟨x, y, hxy.symm⟩, hcommon⟩

```

□

Definition 1.2.2. For $a, b \in \mathbb{Z}$ not both zero, define $\gcd(a, b)$ to be the positive common divisor constructed above from the least positive linear combination. Define $\gcd(0, 0) = 0$.

```

noncomputable def gcd (a b : ℤ) : ℕ :=
  if hab : a ≠ 0 ∨ b ≠ 0 then
    Classical.choose (exists_positive_common_divisor a b hab)
  else 0

```

Theorem 1.2.3. For integers a, b not both zero, $\gcd(a, b)$ is positive, divides both inputs, is an integer linear combination of them, and is divisible by every common divisor.

```

theorem gcd_spec (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) :
  0 < gcd a b ∧ (gcd a b : ℤ) ∣ a ∧ (gcd a b : ℤ) ∣ b ∧
  (∃ x y : ℤ, (gcd a b : ℤ) = a * x + b * y) ∧
  (∀ c : ℤ, c ∣ a → c ∣ b → c ∣ (gcd a b : ℤ))

```

Proof. The definition selects the number constructed above, together with its established properties.

```
rw [gcd, dite_eq_left hab]
exact Classical.choose_spec (exists_positive_common_divisor a b hab)
```

□

Theorem 1.2.4. *Suppose $a, b \in \mathbb{Z}$ are not both zero and $c \in \mathbb{Z}$ divides both. Then $c \leq \gcd(a, b)$.*

```
theorem common_divisor_le_gcd (a b c :  $\mathbb{Z}$ ) (hab :  $a \neq 0 \vee b \neq 0$ )
  (hca :  $c \mid a$ ) (hcb :  $c \mid b$ ) :  $c \leq (\gcd a b : \mathbb{Z})$ 
```

Proof. Write $d = \gcd(a, b) > 0$. Since $c \mid d$, there is $k \in \mathbb{Z}$ with $d = ck$. If $c > 0$, positivity of d forces $k > 0$, hence $k \geq 1$ and $d \geq c$. If $c \leq 0$, then $c < d$ immediately.

```
obtain ⟨hpos, _, _, _, hcommon⟩ := gcd_spec a b hab

have hdpos : (0 :  $\mathbb{Z}$ ) < gcd a b := by
  exact_mod_cast hpos

obtain ⟨k, hk⟩ := hcommon c hca hcb

by_cases hc : 0 < c

· have hkpos : 0 < k := by
  nlinarith

  have hkone : 1 ≤ k := hkpos

  nlinarith

· linarith
```

□

Theorem 1.2.5. *The zero-pair convention is $\gcd(0, 0) = 0$.*

```
theorem gcd_zero_zero : gcd 0 0 = 0
```

Proof. Neither input is nonzero, so the definition assigns the value zero. This is a convention; the common divisors of zero and zero have no greatest integer.

```

unfold gcd
rw [dite_eq_right (by
  rintro (h | h)
  exact h rfl
  exact h rfl)]

```

□

Theorem 1.2.6. *Suppose $a, b \in \mathbb{Z}$ are not both zero and $d \in \mathbb{N}$ divides both. If every integer common divisor is at most d , then $\gcd(a, b) = d$.*

```

theorem gcd_eq_of_greatest (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) (d : ℕ)
  (hda : (d : ℤ) ∣ a) (hdb : (d : ℤ) ∣ b)
  (hgreatest : ∀ c : ℤ, c ∣ a → c ∣ b → c ≤ (d : ℤ)) : gcd a b = d

```

Proof. Since $\gcd(a, b)$ is a common divisor, the hypothesis gives $\gcd(a, b) \leq d$. The preceding bound applied to d gives $d \leq \gcd(a, b)$. Antisymmetry proves equality.

```

have hspec := gcd_spec a b hab
have hle := hgreatest (gcd a b) hspec.2.1 hspec.2.2.1
have hge := common_divisor_le_gcd a b d hab hda hdb
exact_mod_cast (le_antisymm hle hge)

```

□

Theorem 1.2.7. *Suppose $a, b \in \mathbb{Z}$ are not both zero. There are integers x, y such that $\gcd(a, b) = ax + by$.*

```

theorem exists_gcd_eq_mul_add_mul (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) :
  ∃ x y : ℤ, (gcd a b : ℤ) = a * x + b * y

```

Proof. The linear-combination representation is one of the properties established when constructing the gcd.

```

exact (gcd_spec a b hab).2.2.2.1

```

□

Theorem 1.2.8. Suppose $a, b \in \mathbb{Z}$ are not both zero. The set of integer linear combinations $ax + by$ is exactly the set of integer multiples of $\gcd(a, b)$.

```
theorem linear_combinations_eq_gcd_multiples (a b : ℤ) (hab : a ≠ 0 ∨ b
↪ ≠ 0) :
  {z : ℤ | ∃ x y : ℤ, z = a * x + b * y} =
  {z : ℤ | ∃ n : ℤ, z = n * (gcd a b : ℤ)}
```

Proof. Write $d = \gcd(a, b) = ax_0 + by_0$. Since d divides both inputs, choose $u, v \in \mathbb{Z}$ with $a = du$ and $b = dv$. We prove both set inclusions.

```
obtain ⟨x₀, y₀, hbezout⟩ := exists_gcd_eq_mul_add_mul a b hab

have hcommon := (gcd_spec a b hab).2

obtain ⟨u, hu⟩ := hcommon.1
obtain ⟨v, hv⟩ := hcommon.2.1

apply Set.ext
intro z
constructor
```

If $z = ax + by$, then $z = (ux + vy)d$, so z is an integer multiple of d .

```
· rintro ⟨x, y, hz⟩

refine ⟨u * x + v * y, ?_⟩
calc
  z = a * x + b * y := hz
  _ = ((gcd a b : ℤ) * u) * x + ((gcd a b : ℤ) * v) * y := by
    rw [← hu, ← hv]
  _ = (u * x + v * y) * (gcd a b : ℤ) := by
    ring
```

Conversely, if $z = nd$, then $z = a(nx_0) + b(ny_0)$, an integer linear combination of a, b .

```
· rintro ⟨n, hz⟩

refine ⟨n * x₀, n * y₀, ?_⟩
rw [hz, hbezout]
ring
```

□

Theorem 1.2.9. *Let $a, b \in \mathbb{Z}$ be not both zero. They are relatively prime, meaning $\gcd(a, b) = 1$, if and only if there are integers x, y with $1 = ax + by$.*

```
theorem gcd_eq_one_iff_exists_mul_add_mul (a b : ℤ) (hab : a ≠ 0 ∨ b ≠
↪ 0) :
  gcd a b = 1 ↔ ∃ x y : ℤ, 1 = a * x + b * y
```

Proof. For the forward implication, substitute $\gcd(a, b) = 1$ in the previously proved Bezout identity.

```
constructor

· intro h

obtain ⟨x, y, hxy⟩ := exists_gcd_eq_mul_add_mul a b hab

exact ⟨x, y, by simp only [h, Nat.cast_one] using hxy⟩
```

Conversely, 1 divides both inputs. If c is a common divisor, write $a = cu$ and $b = cv$. Then $1 = c(ux + vy)$. For $c > 0$ the second factor is a positive integer, hence at least one, so $c \leq 1$; for $c \leq 0$ the same bound is immediate. The order characterization of the gcd gives the result.

```
· rintro ⟨x, y, hxy⟩
  apply gcd_eq_of_greatest a b hab 1

  · exact ⟨a, by ring⟩
  · exact ⟨b, by ring⟩
  · intro c hca hcb

  obtain ⟨u, hu⟩ := hca
  obtain ⟨v, hv⟩ := hcb

  have hone : 1 = c * (u * x + v * y) := by
    calc
      1 = a * x + b * y := hxy
      _ = c * (u * x + v * y) := by
        rw [hu, hv]
        ring

  by_cases hc : 0 < c

  · have hk : 0 < u * x + v * y := by
      nlinarith
```

```

have hkone : 1 ≤ u * x + v * y := hk

norm_num only [Nat.cast_one]
nlinarith

· norm_num only [Nat.cast_one]
  omega

```

□

Corollary 1.2.10. *For integers a, b not both zero, set $d = \gcd(a, b)$. Then $\gcd(a/d, b/d) = 1$, where the quotients are integers.*

```

theorem gcd_div_gcd_eq_one (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) :
  gcd (a / (gcd a b : ℤ)) (b / (gcd a b : ℤ)) = 1

```

Proof. Since the positive gcd divides both inputs, write $a = du$ and $b = dv$. The integer quotients are exactly u and v , since $d \neq 0$.

```

obtain ⟨hdpos, ⟨u, hu⟩, ⟨v, hv⟩, ⟨x, y, hxy⟩, _⟩ := gcd_spec a b hab

generalize hg : gcd a b = d at *

have hdne : (d : ℤ) ≠ 0 := by
  exact_mod_cast (ne_of_gt hdpos)

have ha : a / (d : ℤ) = u := by
  rw [hu]
  exact Int.mul_ediv_cancel_left u hdne

have hb : b / (d : ℤ) = v := by
  rw [hv]
  exact Int.mul_ediv_cancel_left v hdne

```

Choose Bezout coefficients with $d = ax + by$. Substitution gives $d = d(ux + vy)$, and cancellation of the nonzero integer d yields $1 = ux + vy$.

```

have hone : (1 : ℤ) = u * x + v * y := by
  apply mul_left_cancel₀ hdne
calc
  (d : ℤ) * 1 = a * x + b * y := by
    simp using hxy
  _ = (d : ℤ) * (u * x + v * y) := by
    rw [hu, hv]
    ring

```

The identity rules out $u = v = 0$. Apply the preceding characterization to conclude that the two quotients are relatively prime.

```

have huv : u ≠ 0 v v ≠ 0 := by
  by_contra h
  push Not at h

obtain (rfl, rfl) := h

norm_num at hone

rw [ha, hb]
exact (gcd_eq_one_iff_exists_mul_add_mul u v huv).2 ⟨x, y, hone⟩

```

□

Corollary 1.2.11. *Let $a, b, c \in \mathbb{Z}$. If $\gcd(a, b) = 1$, $a \mid c$, and $b \mid c$, then $ab \mid c$.*

```

theorem mul_dvd_of_coprime (a b c : ℤ) (hab : gcd a b = 1)
  (hac : a ∣ c) (hbc : b ∣ c) : a * b ∣ c

```

Proof. The inputs cannot both be zero because $\gcd(0, 0) = 0$. Thus the coprimality characterization supplies integers x, y with $1 = ax + by$.

```

have hnz : a ≠ 0 v b ≠ 0 := by
  by_contra h
  push Not at h

obtain (rfl, rfl) := h

rw [gcd_zero_zero] at hab
contradiction

obtain ⟨x, y, hxy⟩ := (gcd_eq_one_iff_exists_mul_add_mul a b hnz).1 hab

```

Write $c = ar = bs$. Multiplying the Bezout identity by c and using these two expressions gives $c = acx + bcy = ab(sx + ry)$. The integer $sx + ry$ is the required divisibility witness.

```

obtain ⟨r, hr⟩ := hac

obtain ⟨s, hs⟩ := hbc

refine ⟨s * x + r * y, ?_⟩
calc
  c = c * (a * x + b * y) := by
    rw [← hxy]
    ring
  _ = a * c * x + b * c * y := by
    ring
  _ = a * (b * s) * x + b * (a * r) * y := by
    nth_rw 1 [hs]
    rw [hr]
  _ = (a * b) * (s * x + r * y) := by
    ring

```

□

Theorem 1.2.12. *Let $a, b, c \in \mathbb{Z}$ with $\gcd(a, b) = 1$. If $a \mid bc$, then $a \mid c$ (Euclid's lemma).*

```

theorem dvd_of_dvd_mul_of_coprime (a b c : ℤ) (hab : gcd a b = 1)
  (hdiv : a ∣ b * c) : a ∣ c

```

Proof. As before, coprimality excludes the zero pair, so choose integers x, y with $1 = ax + by$.

```

have hnz : a ≠ 0 ∨ b ≠ 0 := by
  by_contra h
  push Not at h

obtain ⟨rfl, rfl⟩ := h

  rw [gcd_zero_zero] at hab
  contradiction

obtain ⟨x, y, hxy⟩ := (gcd_eq_one_iff_exists_mul_add_mul a b hnz).1 hab

```

Write $bc = ak$. Multiplication by c gives $c = acx + bcy = a(cx + ky)$, which exhibits an integer witness for $a \mid c$.

```

obtain ⟨k, hk⟩ := hdiv

refine ⟨c * x + k * y, ?_⟩
calc
  c = (a * x + b * y) * c := by
    rw [← hxy]
    ring
  _ = a * (c * x) + (b * c) * y := by
    ring
  _ = a * (c * x + k * y) := by
    rw [hk]
    ring

```

□

Theorem 1.2.13. *Let $a, b \in \mathbb{Z}$ be not both zero and let $d \in \mathbb{Z}$ be positive. Then $d = \gcd(a, b)$ if and only if d divides both inputs and every integer common divisor of a, b divides d .*

```

theorem eq_gcd_iff_common_divisor (a b d :  $\mathbb{Z}$ ) (hab :  $a \neq 0 \vee b \neq 0$ )
  (hd :  $0 < d$ ) :
  d = (gcd a b :  $\mathbb{Z}$ ) ↔
    d ∣ a ∧ d ∣ b ∧ (∀ c :  $\mathbb{Z}$ , c ∣ a → c ∣ b → c ∣ d)

```

Proof. If $d = \gcd(a, b)$, all three divisibility properties follow from the construction of the gcd.

```

constructor

· intro heq
  subst d

obtain ⟨_, hda, hdb, _, hcommon⟩ := gcd_spec a b hab

exact ⟨hda, hdb, hcommon⟩

```

Conversely, let $g = \gcd(a, b) > 0$. Because d is a common divisor, $d \leq g$. Since g divides both inputs, the last hypothesis gives $d = gk$. Positivity forces the integer k to be at least one, so $g \leq d$. The two bounds imply $d = g$.

```

· rintro (hda, hdb, hcommon)

have hle := common_divisor_le_gcd a b d hab hda hdb

obtain (hgpos, hga, hgb, _, _) := gcd_spec a b hab
obtain (k, hk) := hcommon (gcd a b) hga hgb

have hgpos' : (0 : ℤ) < gcd a b := by
  exact_mod_cast hgpos

have hkpos : 0 < k := by
  nlinarith

have hkone : 1 ≤ k := hkpos

have hge : (gcd a b : ℤ) ≤ d := by
  nlinarith

exact le_antisymm hle hge

```

□

Lemma 1.2.14. *For all integers a, b , the nonnegative gcd constructed above agrees with Mathlib's standard integer gcd, including $(a, b) = (0, 0)$.*

```

theorem gcd_eq_int_gcd (a b : ℤ) : gcd a b = Int.gcd a b

```

Proof. If the inputs are not both zero, each gcd divides the other by the common-divisor property. Since both are natural numbers, they are equal.

```

by_cases hab : a ≠ 0 ∨ b ≠ 0

· obtain (hda, hdb, _, hgreatest) := gcd_spec a b hab
  apply Nat.dvd_antisymm

  · exact Int.dvd_gcd hda hdb
  · exact Int.natCast_dvd_natCast.mp
    (hgreatest (Int.gcd a b) (Int.gcd_dvd_left a b) (Int.gcd_dvd_right
      ↪ a b))

```

For the zero pair, both definitions give zero. This comparison identifies the representations after the local gcd properties have been established; none of those proofs uses the comparison.

```

• have ha : a = 0 := Classical.byContradiction (fun h => hab (Or.inl h))
  have hb : b = 0 := Classical.byContradiction (fun h => hab (Or.inr h))
  rw [ha, hb, gcd_zero_zero, Int.gcd_zero_left]
  rfl

```

□

1.3 The Euclidean Algorithm

We retain the nonnegative gcd, with $\text{gcd}(0, 0) = 0$. Pairs with the same common divisors have equal gcds: for nonzero pairs, each gcd is at most the other; a zero pair is characterized by having zero as a common divisor.

Lemma 1.3.1. *If $a, b, q, r \in \mathbb{Z}$ and $a = qb + r$, then $\text{gcd}(a, b) = \text{gcd}(b, r)$. No sign or size condition on the remainder is needed for this identity.*

```

theorem gcd_eq_gcd_remainder (a b q r : ℤ) (ha : a = q * b + r) :
  gcd a b = gcd b r

```

Proof. It suffices to compare common divisors. If $a = cu$ and $b = cv$, then $r = c(u - qv)$. Thus a common divisor of a, b also divides b, r .

```

apply gcd_eq_of_common_divisors
intro c
constructor

• rintro ⟨⟨u, hu⟩, ⟨v, hv⟩⟩
  refine ⟨⟨v, hv⟩, ⟨u - q * v, ?_⟩⟩
  calc
    r = a - q * b := by
      linarith [ha]
    _ = c * (u - q * v) := by
      rw [hu, hv]
      ring

```

Conversely, if $b = cu$ and $r = cv$, then $a = c(qu + v)$. Both implications use explicit integer witnesses.

```

• rintro ⟨⟨u, hu⟩, ⟨v, hv⟩⟩
  exact ⟨⟨q * u + v, by
    rw [ha, hu, hv]
    ring⟩, ⟨u, hu⟩⟩

```

□

Lemma 1.3.2. *For all integers a, b , taking absolute values leaves the gcd unchanged: $\gcd(|a|, |b|) = \gcd(a, b)$.*

```
theorem gcd_abs (a b : ℤ) : gcd |a| |b| = gcd a b
```

Proof. An integer c divides $|z|$ exactly when it divides z . For $z \geq 0$ this is immediate; for $z < 0$, negate the multiplication witness. Apply this to both inputs and use the common-divisor characterization.

```
have hdiv (c z : ℤ) : c | |z| ↔ c | z := by
  by_cases hz : 0 ≤ z

  · rw [abs_of_nonneg hz]
  · rw [abs_of_neg (lt_of_not_ge hz)]
    constructor <,> rintro ⟨k, hk⟩ <,> exact ⟨-k, by linarith⟩

exact gcd_eq_of_common_divisors _ _ _ _ (fun c => by rw [hdiv, hdiv])
```

□

Absolute values reduce the computation to nonnegative inputs. No initial ordering is necessary: dividing a smaller input by a larger one simply gives quotient zero. A finite division trace records every quotient and remainder. Its terminal pair is $(g, 0)$; each preceding pair (a, b) satisfies $a = qb + r$ with $0 \leq r < b$, and is followed by (b, r) .

Definition 1.3.3. The relation $\text{EuclideanTrace } a \ b \ g$ records such a finite computation. The natural number types encode nonnegativity, and $r < b$ records strict decrease.

```
inductive EuclideanTrace : ℕ → ℕ → ℕ → Prop
| done (a : ℕ) : EuclideanTrace a 0 a
| step (a b r g : ℕ) (q : ℤ) (hr : r < b)
  (ha : (a : ℤ) = q * (b : ℤ) + (r : ℤ))
  (tail : EuclideanTrace b r g) : EuclideanTrace a b g
```

Theorem 1.3.4. *For every $a, b \in \mathbb{N}$, a finite Euclidean division trace starts at (a, b) and ends at some $(g, 0)$.*

```
theorem euclideanTrace_exists (a b : ℕ) : ∃ g, EuclideanTrace a b g
```

Proof. Use strong induction on the second input, allowing the first input to vary. If the second input is zero, the computation has already ended.

```

induction b using Nat.strong_induction_on generalizing a with
| h b ih =>
  by_cases hb : b = 0

  · subst b
    exact ⟨a, EuclideanTrace.done a⟩

```

Otherwise the previously proved division algorithm supplies $a = qb + r$ with $r < b$. The induction hypothesis gives a finite trace from (b, r) ; prefix the recorded division to obtain the required trace.

```

· obtain ⟨⟨q, r⟩, ⟨ha, hr⟩, _⟩ :=
  existsUnique_quotient_remainder (a : ℤ) (b, Nat.pos_of_ne_zero hb)

obtain ⟨g, hg⟩ := ih r hr b

exact ⟨g, EuclideanTrace.step a b r g q hr ha hg⟩

```

□

Theorem 1.3.5. *If a Euclidean trace starts at (a, b) and ends at $(g, 0)$, then $\gcd(a, b) = g$.*

```

theorem EuclideanTrace.gcd_eq {a b g : ℕ} (h : EuclideanTrace a b g) :
  gcd (a : ℤ) (b : ℤ) = g

```

Proof. Induct on the finite trace. At the terminal pair, $\gcd(a, 0) = a$ for $a \in \mathbb{N}$, by greatestness (and the zero-pair convention). At each earlier division, the remainder lemma preserves the gcd, so the induction hypothesis gives the same terminal value.

```

induction h with
| done a => exact gcd_zero_right a
| step a b r g q hr ha tail ih =>
  exact (gcd_eq_gcd_remainder _ _ q _ ha).trans ih

```

□

Corollary 1.3.6. *If the initial second input b is positive, then the terminal entry g of a Euclidean trace is positive. Hence it is the last nonzero remainder, or the initial divisor if the first remainder is zero.*

```
theorem EuclideanTrace.positive {a b g : ℕ} (h : EuclideanTrace a b g)
  (hb : 0 < b) : 0 < g
```

Proof. The terminal entry equals $\gcd(a, b)$. Since $b > 0$, the inputs are not both zero, and positivity follows from the local gcd specification.

```
rw [← h.gcd_eq]
exact (gcd_spec a b (0r.inr (by exact_mod_cast (ne_of_gt hb)))).1
```

□

Theorem 1.3.7. For integers a, b, k with $k > 0$, we have $\gcd(ka, kb) = k \gcd(a, b)$.

```
theorem gcd_mul_of_pos (a b k : ℤ) (hk : 0 < k) :
  (gcd (k * a) (k * b) : ℤ) = k * (gcd a b : ℤ)
```

Proof. First assume a, b are not both zero. Write $d = \gcd(a, b)$, $a = du$, $b = dv$, and $d = ax + by$. The scaled pair is not both zero, and $kd > 0$.

```
by_cases hab : a ≠ 0 ∨ b ≠ 0
· obtain ⟨hdpos, ⟨u, hu⟩, ⟨v, hv⟩, ⟨x, y, hxy⟩, _⟩ := gcd_spec a b hab
  have hpair : k * a ≠ 0 ∨ k * b ≠ 0 := by
    rcases hab with ha | hb
    · exact 0r.inl (mul_ne_zero (ne_of_gt hk) ha)
    · exact 0r.inr (mul_ne_zero (ne_of_gt hk) hb)
  have hdpos' : (0 : ℤ) < gcd a b := by
    exact_mod_cast hdpos
```

The witnesses u, v show that kd divides both scaled inputs. If c divides them with $ka = cs$ and $kb = ct$, then $kd = k(ax + by) = c(sx + ty)$, so every common divisor divides kd . The local divisibility characterization identifies kd as their gcd.

```

apply Eq.symm
apply (eq_gcd_iff_common_divisor _ _ _ hpair (mul_pos hk hdpow')).2
refine ⟨⟨u, by
  nth_rw 1 [hu]
  ring⟩, ⟨v, by
  nth_rw 1 [hv]
  ring⟩, ?_⟩
rintro c ⟨s, hs⟩ ⟨t, ht⟩
refine ⟨s * x + t * y, ?_⟩
calc
  k * (gcd a b : ℤ) = (k * a) * x + (k * b) * y := by
    rw [hxy]
    ring
  _ = c * (s * x + t * y) := by
    rw [hs, ht]
    ring

```

If both original inputs are zero, both sides vanish by the zero-pair convention.

```

· push Not at hab

obtain ⟨rfl, rfl⟩ := hab

simp [gcd_zero_zero]

```

□

Corollary 1.3.8. *For integers a, b and any nonzero integer k , we have $\gcd(ka, kb) = |k| \gcd(a, b)$.*

```

theorem gcd_mul_of_ne_zero (a b k : ℤ) (hk : k ≠ 0) :
  (gcd (k * a) (k * b) : ℤ) = |k| * (gcd a b : ℤ)

```

Proof. Take absolute values of the scaled inputs, use $|kz| = |k||z|$, and apply the positive-factor result to $|k| > 0$. Finally remove the absolute values from the original gcd.

```

calc
  (gcd (k * a) (k * b) : ℤ) = gcd (|k| * |a|) (|k| * |b|) := by
    rw [← gcd_abs (k * a) (k * b), abs_mul, abs_mul]
  _ = |k| * (gcd |a| |b| : ℤ) := gcd_mul_of_pos _ _ _ (abs_pos.mpr hk)
  _ = |k| * (gcd a b : ℤ) := by
    rw [gcd_abs]

```

□

Lemma 1.3.9. *Any two nonzero integers a, b have a positive natural number that is divisible by both.*

```
theorem exists_positive_common_multiple (a b : ℤ) (ha : a ≠ 0) (hb : b ≠
  0) :
  ∃ m : ℕ, 0 < m ∧ a ∣ (m : ℤ) ∧ b ∣ (m : ℤ)
```

Proof. The integer $(ab)^2$ is positive. Its natural-number representation has the same value, and $(ab)^2 = a(ab^2) = b(a^2b)$ supplies the two divisibility witnesses.

```
have hp : 0 < (a * b) * (a * b) := mul_self_pos.mpr (mul_ne_zero ha hb)

refine ((a * b) * (a * b)).toNat, ?_, ?_, ?_

· omega
· rw [Int.toNat_of_nonneg (le_of_lt hp)]
  exact (a * b * b, by ring)

· rw [Int.toNat_of_nonneg (le_of_lt hp)]
  exact (a * a * b, by ring)
```

□

Definition 1.3.10. For nonzero integers a, b , define $\text{lcm}(a, b)$ to be the least positive common multiple, whose existence was just proved. Extend the definition by zero when either input is zero.

```
noncomputable def lcm (a b : ℤ) : ℕ :=
  if h : a ≠ 0 ∧ b ≠ 0 then
    Nat.find (exists_positive_common_multiple a b h.1 h.2)
  else 0
```

Theorem 1.3.11. *For nonzero integers a, b , the number $m = \text{lcm}(a, b)$ is positive, satisfies $a \mid m$ and $b \mid m$, and obeys $m \leq c$ for every positive integer common multiple c .*

```
theorem lcm_spec (a b : ℤ) (ha : a ≠ 0) (hb : b ≠ 0) :
  0 < lcm a b ∧ a ∣ (lcm a b : ℤ) ∧ b ∣ (lcm a b : ℤ) ∧
  (∀ c : ℤ, a ∣ c → b ∣ c → 0 < c → (lcm a b : ℤ) ≤ c)
```

Proof. The least-element construction supplies positivity and both divisibility properties.

```

classical

rw [lcm, dite_eq_left {ha, hb}]

obtain {hp, hma, hmb} := Nat.find_spec (exists_positive_common_multiple
  ↪ a b ha hb)

refine {hp, hma, hmb, ?_}

```

A positive integer common multiple c is also a positive natural number with the same value. Minimality among natural common multiples gives $m \leq c$, after converting back to integers.

```

intro c hac hbc hc

have hc' : (c.toNat : ℤ) = c := Int.toNat_of_nonneg (le_of_lt hc)

have hmin := Nat.find_min' (exists_positive_common_multiple a b ha hb)
  (show 0 < c.toNat ∧ a | (c.toNat : ℤ) ∧ b | (c.toNat : ℤ) from
    (by omega, by rwa [hc'], by rwa [hc']))

exact_mod_cast (show (Nat.find (exists_positive_common_multiple a b ha
  ↪ hb) : ℤ) ≤ c by
  exact (Int.ofNat_le.mpr hmin).trans_eq hc')

```

□

Theorem 1.3.12. *For positive integers a, b , we have $\gcd(a, b) \operatorname{lcm}(a, b) = ab$.*

```

theorem gcd_mul_lcm (a b : ℤ) (ha : 0 < a) (hb : 0 < b) :
  (gcd a b : ℤ) * (lcm a b : ℤ) = a * b

```

Proof. Put $d = \gcd(a, b) > 0$. Choose $u, v, x, y \in \mathbb{Z}$ such that $a = du, b = dv$, and $d = ax + by$. Since $b > 0$, we have $v > 0$. The positive integer av is a common multiple because $av = bu$.

```

obtain {hd, {u, hu}, {v, hv}, {x, y, hxy}, _} :=
  gcd_spec a b (Or.inl (ne_of_gt ha))

generalize hg : gcd a b = d at *

have hd' : (0 : ℤ) < d := by
  exact_mod_cast hd

```

```

have hvpos : 0 < v := by
  nlinarith [hv]

have hnpos : 0 < a * v := mul_pos ha hvpos

have ham : a | a * v := ⟨v, rfl⟩

have hbm : b | a * v := ⟨u, by
  rw [hu, hv]
  ring⟩

```

Every common multiple c is divisible by av . Indeed, write $c = as = bt$ and multiply the Bezout identity by c : $dc = ab(tx + sy) = d av(tx + sy)$. Cancel the nonzero factor d to obtain the required witness.

```

have hdiv (c : ℤ) (hac : a | c) (hbc : b | c) : a * v | c := by
  obtain ⟨s, hs⟩ := hac
  obtain ⟨t, ht⟩ := hbc

  refine ⟨t * x + s * y, ?_⟩
  apply mul_left_cancel₀ (ne_of_gt hd')
  calc
    (d : ℤ) * c = (a * x + b * y) * c := by
      rw [hxy]
    _ = a * c * x + b * c * y := by
      ring
    _ = a * (b * t) * x + b * (a * s) * y := by
      nth_rw 1 [ht]
      rw [hs]
    _ = (d : ℤ) * (a * v * (t * x + s * y)) := by
      rw [hv]
      ring

```

Let $\ell = \text{lcm}(a, b) > 0$. Minimality gives $\ell \leq av$, while the preceding argument gives $\ell = avk$. Positivity forces the integer $k \geq 1$, so $\ell \geq av$. Therefore $\ell = av$, and $d\ell = dav = ab$.

```

obtain ⟨hlpos, hla, hlb, hleast⟩ := lcm_spec a b (ne_of_gt ha) (ne_of_gt hb)

have hle : (lcm a b : ℤ) ≤ a * v := hleast _ ham hbm hnpos

obtain ⟨k, hk⟩ := hdiv (lcm a b) hla hlb

have hlpos' : (0 : ℤ) < lcm a b := by
  exact_mod_cast hlpos

```

```

have hkpos : 0 < k := by
  nlinarith

have hkone : 1 ≤ k := hkpos

have heq : (lcm a b : ℤ) = a * v := by
  nlinarith

rw [heq, hv]
ring

```

□

Lemma 1.3.13. *For all integers a, b , the lcm constructed above agrees with Mathlib's standard integer lcm, including zero inputs.*

```

theorem lcm_eq_int_lcm (a b : ℤ) : lcm a b = Int.lcm a b

```

Proof. If either input is zero, both definitions give zero.

```

by_cases ha : a = 0

· simp only [ha, lcm, ne_eq, not_true_eq_false, false_and, ↓reduceDite,
  Int.lcm_zero_left]

· by_cases hb : b = 0

  · simp only [hb, lcm, ne_eq, not_true_eq_false, and_false,
    ↪ ↓reduceDite,
    Int.lcm_zero_right]

```

Otherwise the standard lcm is a positive common multiple. It divides every positive common multiple c , so it is at most c . The previously proved uniqueness of the least positive common multiple identifies it with our construction.

```

· apply lcm_eq_of_least a b ha hb (Int.lcm a b)
  (Int.lcm_pos ha hb) (Int.dvd_lcm_left a b) (Int.dvd_lcm_right a b)
  intro c hac hbc hc
  apply Int.le_of_dvd hc
  have hc' : (c.toNat : ℤ) = c := Int.toNat_of_nonneg hc.le
  rw [← hc']
  apply Int.natCast_dvd_natCast.mpr

```

```
exact Int.lcm_dvd (by rwa [hc']) (by rwa [hc'])
```

□

1.4 The Diophantine Equation $ax+by=c$

All coefficients and unknowns are integers. We use the nonnegative gcd constructed above. The Lean namespace is `ElementaryNumberTheory.Chapter01`.

Theorem 1.4.1. *The equation $ax+by = c$ has an integer solution if and only if $\gcd(a, b) \mid c$. This includes $a = b = 0$, when both conditions mean $c = 0$.*

```
theorem exists_mul_add_mul_eq_iff (a b c : ℤ) :
  (∃ x y : ℤ, a * x + b * y = c) ↔ (gcd a b : ℤ) ∣ c
```

Proof. First suppose that the coefficients are not both zero. Write $a = du$, $b = dv$, and $d = ar + bs$, where $d = \gcd(a, b) > 0$.

```
by_cases hab : a ≠ 0 ∨ b ≠ 0

· obtain ⟨_, ⟨u, hu⟩, ⟨v, hv⟩, ⟨r, s, hrs⟩, _⟩ := gcd_spec a b hab
  generalize hg : gcd a b = d at *
  constructor
```

Any solution gives $c = ax + by = d(ux + vy)$, so $d \mid c$.

```
· rintro ⟨x, y, rfl⟩
  refine ⟨u * x + v * y, ?_⟩
  rw [hu, hv]
  ring
```

Conversely, if $c = dk$, multiplying the Bezout identity by k gives the solution $x = rk$, $y = sk$.

```
· rintro ⟨k, rfl⟩
  refine ⟨r * k, s * k, ?_⟩
  rw [hrs]
  ring
```

If both coefficients vanish, their gcd is zero. The equation is solvable exactly when $c = 0$, which is also the meaning of $0 \mid c$.

```

• have ha : a = 0 := Classical.byContradiction (fun h => hab (Or.inl h))
  have hb : b = 0 := Classical.byContradiction (fun h => hab (Or.inr h))
  simp [ha, hb, gcd_zero_zero, eq_comm]

```

□

Theorem 1.4.2. Suppose a, b are not both zero, put $d = \gcd(a, b)$, and fix a solution (x_0, y_0) of $ax + by = c$. An integer pair (x, y) is a solution if and only if, for some $t \in \mathbb{Z}$,

$$x = x_0 + \frac{b}{d}t, \quad y = y_0 - \frac{a}{d}t.$$

The quotients are integers since d divides both coefficients.

```

theorem mul_add_mul_eq_iff (a b c x_0 y_0 x y : ℤ) (hab : a ≠ 0 ∨ b ≠ 0)
  (h_0 : a * x_0 + b * y_0 = c) :
  a * x + b * y = c ↔ ∃ t : ℤ,
    x = x_0 + (b / (gcd a b : ℤ)) * t ∧
    y = y_0 - (a / (gcd a b : ℤ)) * t

```

Proof. Choose u, v, r, s with $a = du$, $b = dv$, and $d = ar + bs$. Since $d > 0$, the exact quotients a/d and b/d are u and v .

```

obtain ⟨hd, ⟨u, hu⟩, ⟨v, hv⟩, ⟨r, s, hrs⟩, _⟩ := gcd_spec a b hab
generalize hg : gcd a b = d at *

have hdne : (d : ℤ) ≠ 0 := by
  exact_mod_cast (ne_of_gt hd)

have ha : a / (d : ℤ) = u := by
  rw [hu]
  exact Int.mul_ediv_cancel_left u hdne

have hb : b / (d : ℤ) = v := by
  rw [hv]
  exact Int.mul_ediv_cancel_left v hdne

```

Substituting into the Bezout identity and cancelling d gives $ur + vs = 1$.

```

have hone : u * r + v * s = 1 := by
  apply mul_left_cancel₀ hdne
calc
  (d : ℤ) * (u * r + v * s) = a * r + b * s := by
    rw [hu, hv]
    ring
  _ = (d : ℤ) * 1 := by linarith [hrs]

rw [ha, hb]
constructor

```

For another solution, subtract the two equations and cancel d . With $X = x - x_0$ and $Y = y - y_0$, this gives $uX + vY = 0$.

```

· intro h
have hdiff : u * (x - x₀) + v * (y - y₀) = 0 := by
  apply mul_left_cancel₀ hdne
calc
  (d : ℤ) * (u * (x - x₀) + v * (y - y₀)) =
    (a * x + b * y) - (a * x₀ + b * y₀) := by
      rw [hu, hv]
      ring
  _ = (d : ℤ) * 0 := by
    rw [h, h₀]
    ring

```

Put $t = sX - rY$. Then $(ur + vs)X - r(uX + vY) = vt$ and $(ur + vs)Y - s(uX + vY) = -ut$. Hence $X = vt$ and $Y = -ut$, as required.

```

refine (s * (x - x₀) - r * (y - y₀), ?_, ?_)

· nlinarith [congrArg (fun z => z * (x - x₀)) hone,
  congrArg (fun z => z * r) hdiff]
· nlinarith [congrArg (fun z => z * (y - y₀)) hone,
  congrArg (fun z => z * s) hdiff]

```

Conversely, substitution of $x = x_0 + vt$ and $y = y_0 - ut$ cancels the terms involving t , because $av = bu$. The result is $ax_0 + by_0 = c$.

```

· rintro ⟨t, rfl, rfl⟩
calc
  a * (x₀ + v * t) + b * (y₀ - u * t) = a * x₀ + b * y₀ := by
    rw [hu, hv]
    ring
  _ = c := h₀

```

□

Corollary 1.4.3. *If $\gcd(a, b) = 1$ and $ax_0 + by_0 = c$, then the integer solutions are exactly $x = x_0 + bt$, $y = y_0 - at$ for $t \in \mathbb{Z}$.*

```
theorem mul_add_mul_eq_iff_of_coprime (a b c x₀ y₀ x y : ℤ)
  (hab : gcd a b = 1) (h₀ : a * x₀ + b * y₀ = c) :
  a * x + b * y = c ↔ ∃ t : ℤ, x = x₀ + b * t ∧ y = y₀ - a * t
```

Proof. The coefficients cannot both be zero, since $\gcd(0, 0) = 0$.

```
have hnz : a ≠ 0 ∨ b ≠ 0 := by
  by_contra h
  push Not at h
  obtain (rfl, rfl) := h
  rw [gcd_zero_zero] at hab
  contradiction
```

Apply the general solution formula and substitute $d = 1$.

```
simpa only [hab, Nat.cast_one, Int.ediv_one] using
  mul_add_mul_eq_iff a b c x₀ y₀ x y hnz h₀
```

□

Chapter 2

Primes and Their Distribution

2.1 The Fundamental Theorem of Arithmetic

Positive integers are represented by natural numbers with explicit positivity hypotheses. Integer factors may still be negative or zero. The Lean declarations in this section belong to `ElementaryNumberTheory.Chapter02`.

Definition 2.1.1. A natural number $p > 1$ is *prime* when every natural divisor is 1 or p . Since $p > 0$, zero cannot divide p ; thus this is precisely the condition on positive divisors.

```
def isPrime (p : ℕ) : Prop :=  
  1 < p ∧ ∀ d : ℕ, d ∣ p → d = 1 ∨ d = p
```

Definition 2.1.2. A natural number $n > 1$ which is not prime is *composite*. In particular, neither zero nor one is prime or composite.

```
def isComposite (n : ℕ) : Prop :=  
  1 < n ∧ ¬isPrime n
```

Lemma 2.1.3. *The divisor definition of primality agrees with the standard predicate `Nat.Prime`.*

```
theorem isPrime_iff_nat_prime (p : ℕ) : isPrime p ↔ Nat.Prime p
```

Proof. The standard divisor characterization requires $2 \leq p$ and that every divisor be 1 or p . For natural numbers $2 \leq p$ is the same condition as $1 < p$.

```
rw [Nat.prime_def]
rfl
```

□

Theorem 2.1.4. *Let p be prime and let $a, b \in \mathbb{Z}$. If $p \mid ab$, then $p \mid a$ or $p \mid b$.*

```
theorem prime_dvd_mul (p : ℕ) (hp : isPrime p) (a b : ℤ)
  (h : (p : ℤ) ∣ a * b) : (p : ℤ) ∣ a ∨ (p : ℤ) ∣ b
```

Proof. If $p \mid a$, the assertion is immediate. Otherwise, p is nonzero because $p > 1$.

```
classical
by_cases hpa : (p : ℤ) ∣ a

· exact Or.inl hpa

· have hnz : (p : ℤ) ≠ 0 := by
  have := hp.1
  omega
```

The positive integer $d = \gcd(p, a)$ divides p , so it is 1 or p . The second possibility would imply $p \mid a$, contrary to the assumption. Thus $\gcd(p, a) = 1$.

```
obtain (_, hdp, hda, _, _) := Chapter01.gcd_spec (p : ℤ) a (Or.inl hnz)
have hd : Chapter01.gcd (p : ℤ) a ∣ p := Int.natCast_dvd_natCast.mp hdp
have hcoprime : Chapter01.gcd (p : ℤ) a = 1 := by
  rcases hp.2 _ hd with hone | heq

· exact hone
· rw [heq] at hda
  exact (hpa hda).elim
```

The previously proved Euclid lemma for coprime integers now gives $p \mid b$.

```
exact Or.inr (Chapter01.dvd_of_dvd_mul_of_coprime (p : ℤ) a b hcoprime
  ↪ h)
```

□

Corollary 2.1.5. *If a prime p divides the product of a finite list of integers, then it divides at least one member of that list.*

```
theorem prime_dvd_list_prod (p : ℕ) (hp : isPrime p) (l : List ℤ)
  (h : (p : ℤ) ∣ l.prod) : ∃ a ∈ l, (p : ℤ) ∣ a
```

Proof. Induct on the list. The empty product is 1. If $1 = pk$, then $p > 1$ forces $k > 0$, whence $pk \geq p > 1$, a contradiction.

```
induction l with
| nil =>
  obtain ⟨k, hk⟩ := h
  have hpos : (1 : ℤ) < p := by exact_mod_cast hp.1
  change 1 = (p : ℤ) * k at hk
  have hkpos : 0 < k := by nlinarith
  nlinarith
```

For a nonempty list with first entry a , the preceding theorem says that p divides a or the product of the remaining entries. In the latter case apply the induction hypothesis.

```
| cons a l ih =>
  rcases prime_dvd_mul p hp a l.prod h with ha | hl

  · exact ⟨a, List.mem_cons_self, ha⟩
  · obtain ⟨b, hb, hpb⟩ := ih hl
    exact ⟨b, List.mem_cons_of_mem a hb, hpb⟩
```

□

Lemma 2.1.6. *If p is prime and $a, b \in \mathbb{N}$, then $p \mid ab$ implies $p \mid a$ or $p \mid b$.*

```
theorem prime_dvd_mul_nat (p a b : ℕ) (hp : isPrime p) (h : p ∣ a * b) :
  p ∣ a ∨ p ∣ b
```

Proof. View the natural numbers as integers and apply the product theorem. Divisibility of natural numbers is unchanged by this inclusion, so the conclusion returns to $\mathbb{N}$.

```
have h' : (p : ℤ) ∣ (a : ℤ) * (b : ℤ) := by exact_mod_cast h
rcases prime_dvd_mul p hp a b h' with ha | hb

  · exact 0r.inl (Int.natCast_dvd_natCast.mp ha)
  · exact 0r.inr (Int.natCast_dvd_natCast.mp hb)
```

□

Corollary 2.1.7. *If p and all members of a finite list $q_1, \dots, q_n$ are prime, and $p \mid q_1 \cdots q_n$, then $p = q_k$ for some $1 \leq k \leq n$. List membership expresses this conclusion without choosing an indexing convention.*

```
theorem prime_eq_of_dvd_prod (p : ℕ) (hp : isPrime p) (l : List ℕ)
  (hl : ∀ q ∈ l, isPrime q) (h : p ∣ l.prod) : p ∈ l
```

Proof. Induct on the list. As before, a prime cannot divide the empty product 1.

```
induction l with
| nil =>
  obtain ⟨k, hk⟩ := h
  change 1 = p * k at hk
  have hkpos : 0 < k := by nlinarith [hp.1]
  nlinarith [hp.1]
```

For a list beginning with q , either $p \mid q$ or p divides the remaining product. In the first case the divisors of the prime q give $p = 1$ or $p = q$. Since $p > 1$, only $p = q$ is possible.

```
| cons q l ih =>
  rcases prime_dvd_mul_nat p q l.prod hp h with hq | ht

  · rcases (hl q List.mem_cons_self).2 p hq with hone | heq

    · have := hp.1
      omega
    · exact List.mem_cons.mpr (Or.inl heq)
```

In the second case apply the induction hypothesis to the remaining prime entries.

```
· exact List.mem_cons.mpr
  (Or.inr (ih (fun r hr => hl r (List.mem_cons_of_mem q hr)) ht))
```

□

A list records factors with their multiplicities. Its product is denoted by `List.prod`; the empty product is 1. Two lists are permutations of each other when they have the same entries with the same multiplicities, possibly in different orders.

Lemma 2.1.8. *Every natural number $n > 0$ is the product of a finite list of primes; for $n = 1$ the list may be empty.*

```
theorem exists_prime_factors (n : ℕ) (hn : 0 < n) :
  ∃ l : List ℕ, (∀ p ∈ l, isPrime p) ∧ l.prod = n
```

Proof. Use strong induction on n . The empty list has product 1. If $n > 1$ is prime, the one-element list containing n has the required product.

```
classical
induction n using Nat.strong_induction_on with
| h n ih =>
  by_cases hone : n = 1

  · exact ([], by simp, by simpa using hone.symm)

  · have hgt : 1 < n := by omega
    by_cases hp : isPrime n

    · exact ([n], by simp, by simp)
```

If $n > 1$ is not prime, it has a divisor d different from 1 and n . Write $n = dk$. Positivity gives $d, k > 0$. If $d \geq n$, then $k = 1$, contradicting $d \neq n$; hence $d < n$. Also $d \geq 2$, so $k < n$.

```
· have hdiv : ∃ d : ℕ, d | n ∧ d ≠ 1 ∧ d ≠ n := by
  simp only [isPrime, hgt, true_and, not_forall, not_or, exists_prop]
  ↪ using hp

obtain (d, (k, hk), hdl, hdn) := hdiv
have hdpos : 0 < d := by nlinarith
have hkpos : 0 < k := by nlinarith
have hdl : d < n := by
  by_contra h
  have hk1 : k = 1 := by nlinarith
  simp [hk1] at hk
  exact hdn hk.symm

have hkl : k < n := by
  have hd2 : 2 ≤ d := by omega
  nlinarith
```

By the induction hypothesis, d and k have prime factor lists. Concatenating them preserves primality of every entry, and its product is $dk = n$.

```

obtain ⟨l, hl, heql⟩ := ih d hdlt hdpos
obtain ⟨m, hm, heqm⟩ := ih k hklt hkpos
refine ⟨l ++ m, ?_, ?_⟩

· intro p hp
  rcases List.mem_append.mp hp with hpl | hpm

  · exact hl p hpl
  · exact hm p hpm

· rw [List.prod_append, heql, heqm, ← hk]

```

□

Lemma 2.1.9. *Two finite lists of primes with the same product are permutations of one another.*

```

theorem prime_factors_perm (l m : List ℕ) (hl : ∀ p ∈ l, isPrime p)
  (hm : ∀ p ∈ m, isPrime p) (hprod : l.prod = m.prod) : l.Perm m

```

Proof. Induct on the first list. If it is empty, its product is 1. The second list cannot begin with a prime $q > 1$, since $1 = qM$ would imply $M > 0$ and hence $qM > 1$. Thus both lists are empty.

```

classical
induction l generalizing m with
| nil =>
  cases m with
  | nil => exact List.Perm.refl []
  | cons q m =>
    have hq := (hm q List.mem_cons_self).1
    change 1 = q * m.prod at hprod
    have hnpos : 0 < m.prod := by nlinarith
    nlinarith

```

If the first list begins with p , then p divides the second product. The preceding corollary ensures that p occurs in the second list.

```

| cons p l ih =>
  have hp := hl p List.mem_cons_self
  have hpm : p ∈ m := by
    apply prime_eq_of_dvd_prod p hp m hm
    exact ⟨l.prod, hprod.symm⟩

```

Move one occurrence of p to the front of the second list and remove it. Since $p > 0$, cancellation shows that the two remaining products are equal.

```

have hperm : m.Perm (p :: m.erase p) := List.perm_cons_erase hpm
have htail : l.prod = (m.erase p).prod := by
  have heq := hperm.prod_eq
  simp only [List.prod_cons] at hprod heq
  exact Nat.eq_of_mul_eq_mul_left hp.1.le (hprod.trans heq)

```

The remaining lists still consist of primes, so the induction hypothesis makes them permutations. Restoring the removed occurrence of p gives the desired permutation of the original lists.

```

have htailperm : l.Perm (m.erase p) := ih (m.erase p)
  (fun q hq => hl q (List.mem_cons_of_mem p hq))
  (fun q hq => hm q (List.mem_of_mem_erase hq)) htail

exact (List.Perm.cons p htailperm).trans hperm.symm

```

□

Theorem 2.1.10. *Every natural number $n > 1$ is a prime or a product of primes. Its prime factor list is nonempty and is unique up to permutation; a list with one entry represents the prime case.*

```

theorem fundamental_theorem_of_arithmetic (n : ℕ) (hn : 1 < n) :
  ∃ l : List ℕ, l ≠ [] ∧ (∀ p ∈ l, isPrime p) ∧ l.prod = n ∧
  ∀ m : List ℕ, (∀ p ∈ m, isPrime p) → m.prod = n → m.Perm l

```

Proof. The existence lemma gives a prime factor list for n . It cannot be empty, since the empty product is 1 and $n > 1$.

```

obtain {l, hl, hprod} := exists_prime_factors n (by omega)
refine {l, ?_, hl, hprod, ?_}

· intro heq
  subst l
  simp only [List.prod_nil] at hprod
  omega

```

Any other prime factor list for n has the same product, so the uniqueness lemma makes it a permutation of this list.

```

• intro m hm hmn
  exact prime_factors_perm m l hm hl (hmn.trans hprod.symm)

```

□

To group equal prime factors, use a finitely supported function $e: \mathbb{N} \rightarrow \mathbb{N}$. Its support is the finite set of p with $e(p) \neq 0$; these and only these indices have positive exponents. In Lean this type is `Finsupp`, and its product means $\prod_{p \in \text{supp}(e)} p^{e(p)}$. A multiset forgets the order of a list while retaining its multiplicities. Passing between multisets and exponent functions only groups or repeats entries; it does not invoke a factorization result.

Lemma 2.1.11. *For every $n > 0$ there is a unique finitely supported function $e: \mathbb{N} \rightarrow \mathbb{N}$ whose support consists of primes and for which $\prod_{p \in \text{supp}(e)} p^{e(p)} = n$.*

```

theorem existsUnique_prime_exponents (n : ℕ) (hn : 0 < n) :
  ∃! e : ℕ →0 ℕ, (∀ p ∈ e.support, isPrime p) ∧
    e.prod (fun p k => p ^ k) = n

```

Proof. Choose a prime factor list and let $e(p)$ count its occurrences of p . Every member of the support is prime. Grouping repeated factors replaces their product by the product of prime powers, still equal to n .

```

classical
obtain {l, hl, hprod} := exists_prime_factors n hn
let e := (l : Multiset ℕ).toFinsupp
have he : (∀ p ∈ e.support, isPrime p) ∧ e.prod (fun p k => p ^ k) = n
  ↪ := by
  constructor

  • intro p hp
    exact hl p (by simp [e] using hp)

  • rw [← Finsupp.prod_toMultiset, Multiset.toFinsupp_toMultiset]
    exact hprod

```

For any other exponent function f , repeat each supported prime p exactly $f(p)$ times to form a list. All its entries are prime, and its product is the specified power product n .

```

refine (e, he, ?_)
rintro f (hf, hfn)
let m := f.toMultiset.toList
have hm : ∀ p ∈ m, isPrime p := by
  intro p hp
  exact hf p (by simp [m] using hp)

have hmprod : m.prod = n := by
  rw [Multiset.prod_toList, Finsupp.prod_toMultiset]
  exact hfn

```

The uniqueness lemma makes the two expanded lists permutations. Their multisets, and hence all their multiplicities, agree. Thus $f = e$.

```

have hperm := prime_factors_perm m l hm hl (hmprod.trans hprod.symm)
have hmulti : f.toMultiset = (l : Multiset ℕ) := by
  simp [m] using Multiset.coe_eq_coe.mpr hperm

have heq := congrArg Multiset.toFinsupp hmulti
simp [e] using heq

```

□

Definition 2.1.12. A *canonical factorization* of n is a pair (L, e) : the list L is strictly increasing, its entries are exactly the support of the finitely supported exponent function e , every supported entry is prime, and the product of the corresponding prime powers is n . The notation *Pairwise* with $<$ requires every earlier entry to be less than every later entry.

```

def isCanonicalFactorization (n : ℕ) (r : List ℕ × (ℕ → ℕ)) : Prop :=
  r.1.Pairwise (· < ·) ∧ r.1.toFinset = r.2.support ∧
  (∀ p ∈ r.2.support, isPrime p) ∧ r.2.prod (fun p k => p ^ k) = n

```

Corollary 2.1.13. Every natural number $n > 1$ has exactly one canonical factorization (L, e) .

```

theorem existsUnique_canonical_factorization (n : ℕ) (hn : 1 < n) :
  ∃! r : List ℕ × (ℕ → ℕ), isCanonicalFactorization n r

```

Proof. Take the unique exponent function and arrange its finite support in increasing order. The support has no repeated elements, so the resulting list is strictly increasing.

```

classical
obtain ⟨e, he, huniq⟩ := existsUnique_prime_exponents n (by omega)
let l := e.support.sort (· ≤ ·)
have hl : l.Pairwise (· < ·) :=
  List.sortedLT_iff_pairwise.mp (Finset.sortedLT_sort e.support)

```

This pair is a canonical factorization. For any other pair (M, f) , uniqueness of prime exponents gives $f = e$.

```

refine ⟨(l, e), (hl, Finset.sort_toFinset _ _, he), ?_⟩
rintro ⟨m, f⟩ ⟨hm, hsupport, hf⟩
have hfe : f = e := huniq f hf
subst f

```

The list M is strictly increasing and has the same underlying finite set as L . Sorting this finite set therefore returns M as well as L , so the lists agree and the pairs are equal.

```

have hsorted : e.support.sort (· ≤ ·) = m := by
  rw [← hsupport]
  exact (List.toFinset_sort (· ≤ ·) hm.nodup).mpr
    (hm.imp (fun h => Nat.le_of_lt h))

exact Prod.ext hsorted.symm rfl

```

□

Corollary 2.1.14. *If (L, e) is the canonical factorization of $n > 1$, then L is nonempty. Writing $L = (p_1, \dots, p_r)$ and $k_i = e(p_i)$, we have*

$$n = p_1^{k_1} \cdots p_r^{k_r}, \quad p_1 < \cdots < p_r, \quad k_i > 0,$$

and each p_i is prime. Together with the preceding uniqueness result, this is the canonical prime-power form.

```

theorem canonical_factorization_spec (n : ℕ) (hn : 1 < n)
  (r : List ℕ × (ℕ → ℕ)) (hr : isCanonicalFactorization n r) :
  r.1 ≠ [] ∧ r.1.Pairwise (· < ·) ∧
  (∀ p ∈ r.1, isPrime p ∧ 0 < r.2 p) ∧
  (r.1.map (fun p => p ^ r.2 p)).prod = n

```

Proof. The list has distinct entries and its underlying set is the support. Thus the product along the list equals the product over the support, which is n .

```

obtain (hsorted, hsupport, hprime, hprod) := hr
have hproduct : (r.1.map (fun p => p ^ r.2 p)).prod = n := by
  rw [← List.prod_toFinset _ hsorted.nodup, hsupport]
  exact hprod

refine {?_, hsorted, ?_, hproduct}

```

If the list were empty, this product would be 1, contrary to $n > 1$.

```

· intro hn1l
  rw [hn1l] at hproduct
  simp only [List.map_nil, List.prod_nil] at hproduct
  omega

```

Every listed prime belongs to the support. By definition of the support its exponent is nonzero, hence positive; primality is part of the canonical factorization condition.

```

· intro p hp
have hmem : p ∈ r.2.support := by
  rw [← hsupport]
  exact List.mem_toFinset.mpr hp

exact (hprime p hmem, Nat.pos_of_ne_zero (Finsupp.mem_support_iff.mp
  ↪ hmem))

```

□

2.2 The Sieve of Eratosthenes

We continue to represent positive integers by natural numbers with explicit positivity assumptions. The Lean namespace is `ElementaryNumberTheory.Chapter02`.

Lemma 2.2.1. *Every natural number $n > 1$ has a prime divisor $p \leq n$.*

```

theorem exists_prime_divisor (n : ℕ) (hn : 1 < n) :
  ∃ p : ℕ, isPrime p ∧ p | n ∧ p ≤ n

```

Proof. The fundamental theorem of arithmetic supplies a nonempty list of prime factors. Take its first entry p and write $n = pM$, where M is the product of the remaining entries. Since $n > 0$, we have $M \geq 1$, so $p \leq n$.

```

obtain ⟨l, hne, hl, hprod, _⟩ := fundamental_theorem_of_arithmetic n hn
cases l with
| nil => exact (hne rfl).elim
| cons p l =>
  have hp := hl p List.mem_cons_self
  have heq : n = p * l.prod := hprod.symm
  have hpos : 0 < l.prod := by
    nlinarith
  exact ⟨p, hp, ⟨l.prod, heq⟩, by nlinarith⟩

```

□

Theorem 2.2.2. *If $n > 1$ is composite, it has a prime divisor p satisfying $p \leq \sqrt{n}$. For integer p , this is equivalent to $p \leq \lfloor \sqrt{n} \rfloor$; Lean uses the natural square root, characterized by $p \leq \text{sqrt}(n)$ if and only if $p^2 \leq n$.*

```

theorem exists_prime_divisor_le_sqrt (n : ℕ) (hn : isComposite n) :
  ∃ p : ℕ, isPrime p ∧ p | n ∧ p ≤ Nat.sqrt n

```

Proof. Since n is composite, choose a divisor d other than 1 and n , and write $n = dk$. Positivity and the proper-divisor condition give $d > 1$ and $k > 1$.

```

classical
have hdiv : ∃ d : ℕ, d | n ∧ d ≠ 1 ∧ d ≠ n := by
  simpa only [isPrime, hn.1, true_and, not_forall, not_or, exists_prop]
  ↪ using hn.2

obtain ⟨d, ⟨k, hk⟩, hd1, hdn⟩ := hdiv
have hdpos : 0 < d := by
  nlinarith [hn.1]
have hd : 1 < d := by
  omega
have hkgt : 1 < k := by
  by_contra h
  have hk1 : k = 1 := by
    nlinarith [hn.1]
  simp only [hk1, mul_one] at hk
  exact hdn hk.symm

```

Choose a prime divisor p of $\min(d, k)$. If p divides d , write $d = pt$ and obtain $n = p(tk)$. If it divides k , write $k = pt$ and obtain $n = p(dt)$. Thus $p \mid n$ in either case.

```

obtain (p, hp, (t, ht), hle) := exists_prime_divisor (min d k) (by
  ↪ omega)
refine (p, hp, ?_, Nat.le_sqrt.mpr ?_)

· rcases le_total d k with hdk | hkd

· rw [min_eq_left hdk] at ht
  refine (t * k, ?_)
  rw [hk, ht]
  ring

· rw [min_eq_right hkd] at ht
  refine (d * t, ?_)
  rw [hk, ht]
  ring

```

The choice of p gives $p \leq d$ and $p \leq k$. Therefore $p^2 \leq dk = n$, proving the square-root bound.

```

· have hpd : p ≤ d := le_trans hle (min_le_left _ _)
  have hpk : p ≤ k := le_trans hle (min_le_right _ _)
  nlinarith [Nat.mul_le_mul hpd hpk]

```

□

Consequently, a sieve need only test prime divisors up to the square root: a number greater than one with no such prime divisor cannot be composite.

Lemma 2.2.3. *For natural numbers k, n with $0 < k \leq n$, there is a natural number t such that $n! = kt$.*

```

theorem factorial_multiple (n k : ℕ) (hk : 0 < k) (hkn : k ≤ n) :
  ∃ t : ℕ, n.factorial = k * t

```

Proof. Induct on n . The case $n = 0$ has no admissible k . At the next step, if $k = n + 1$, use $(n + 1)! = (n + 1)n!$. Otherwise $k \leq n$, and multiplying the inductive equality $n! = kt$ by $n + 1$ gives the required witness.

```

induction n with
| zero => omega
| succ n ih =>
  by_cases heq : k = n + 1

· exact (n.factorial, by rw [heq, Nat.factorial_succ])
· obtain (t, ht) := ih (by omega)

```

```

refine (n + 1) * t, ?_
rw [Nat.factorial_succ, ht]
ring

```

□

Lemma 2.2.4. *If a prime p divides $M + 1$, it does not divide M .*

```

theorem not_dvd_of_dvd_add_one (p M : ℕ) (hp : isPrime p) (h : p ∣ M +
↪ 1) :
  ¬p ∣ M

```

Proof. If $M = pj$ and $M + 1 = pk$, then $j < k$. Hence $M + p = p(j + 1) \leq pk = M + 1$, implying $p \leq 1$, a contradiction.

```

rintro ⟨j, hj⟩
obtain ⟨k, hk⟩ := h
have hjk : j + 1 ≤ k := by
  nlinarith [hp.1]
nlinarith [Nat.mul_le_mul_left p hjk, hp.1]

```

□

Lemma 2.2.5. *For every natural number m , some prime exceeds m .*

```

theorem exists_prime_gt (m : ℕ) : ∃ p : ℕ, isPrime p ∧ m < p

```

Proof. The number $m! + 1$ exceeds one, so it has a prime divisor p .

```

obtain ⟨p, hp, hdiv, _⟩ :=
  exists_prime_divisor (m.factorial + 1) (by
    have := Nat.factorial_pos m
    omega)

```

If $p \leq m$, the factorial lemma gives $p \mid m!$. This contradicts the fact that a prime cannot divide both $m!$ and $m! + 1$. Thus $p > m$.

```

refine (p, hp, ?_)
by_contra h
obtain (t, ht) := factorial_multiple m p (lt_trans Nat.zero_lt_one hp.1)
  ↪ (by omega)
exact not_dvd_of_dvd_add_one p m.factorial hp hdiv (t, ht)

```

□

Theorem 2.2.6. *There are infinitely many prime numbers.*

```

theorem infinite_primes : Set.Infinite {p : ℕ | isPrime p}

```

Proof. Every finite set of natural numbers is bounded. The preceding lemma produces a prime beyond any proposed bound, so the set of primes is infinite.

```

apply Set.infinite_of_forall_exists_gt
intro m
obtain (p, hp, hgt) := exists_prime_gt m
exact (p, hp, hgt)

```

□

List the primes in increasing order as $p_1, p_2, \dots$. The general enumeration Nat.nth starts at zero, so $\text{Nat.nth isPrime } i$ denotes p_{i+1} . The infinitude just proved ensures that this enumeration always gives a prime, is increasing, and contains every prime. Put $P_0 = 1$ and $P_n = p_1 \cdots p_n$ for $n > 0$.

Lemma 2.2.7. *For every $n \geq 0$, the next prime satisfies $p_{n+1} \leq P_n + 1$.*

```

theorem nth_prime_le_prod_add_one (n : ℕ) :
  Nat.nth isPrime n ≤ (∏ i ∈ Finset.range n, Nat.nth isPrime i) + 1

```

Proof. All factors in P_n are positive, and the empty product is also positive. Thus $P_n + 1 > 1$.

```

classical
let P :=  $\prod$  i  $\in$  Finset.range n, Nat.nth isPrime i
have hP : 0 < P := by
  apply Finset.prod_pos
  intro i _
  exact lt_trans Nat.zero_lt_one (Nat.nth_mem_of_infinite
     $\hookrightarrow$  infinite_primes i).1

```

Choose a prime divisor $q \leq P_n + 1$ and let its enumeration index be k , so $q = p_{k+1}$. If $k < n$, then q is a factor of P_n , contradicting the consecutive-number lemma. Therefore $k \geq n$.

```

obtain (q, hq, hdiv, hqle) := exists_prime_divisor (P + 1) (by omega)
obtain (k, hk) := Nat.subset_range_nth (p := isPrime) (show q  $\in$ 
 $\hookrightarrow$  Set.ofPred isPrime from hq)
have hnk : n  $\leq$  k := by
  by_contra h
  have hmem : k  $\in$  Finset.range n := Finset.mem_range.mpr (by omega)
  have hfactor := Finset.prod_erase_mul (Finset.range n) (Nat.nth
     $\hookrightarrow$  isPrime) hmem
  have hqP : q  $\mid$  P := by
    refine ( $\prod$  i  $\in$  (Finset.range n).erase k, Nat.nth isPrime i, ?_)
    dsimp [P]
    rw [ $\leftarrow$  hfactor, hk, mul_comm]

exact not_dvd_of_dvd_add_one q P hq hdiv hqP

```

Increasing order gives $p_{n+1} \leq p_{k+1} = q \leq P_n + 1$.

```

calc
  Nat.nth isPrime n  $\leq$  Nat.nth isPrime k := Nat.nth_monotone
 $\hookrightarrow$  infinite_primes hnk
_ = q := hk
_  $\leq$  P + 1 := hqle

```

□

Lemma 2.2.8. *For every $n \geq 0$, we have $P_n < 2^{2^n}$.*

```

theorem prod_initial_primes_lt (n :  $\mathbb{N}$ ) :
  ( $\prod$  i  $\in$  Finset.range n, Nat.nth isPrime i) < 2 ^ (2 ^ n)

```

Proof. Induct on n . For $n = 0$, the assertion is $1 < 2$. If $P_n < B = 2^{2^n}$, integrality gives $P_n + 1 \leq B$, so the preceding bound yields $p_{n+1} \leq B$.

```

induction n with
| zero => simp
| succ n ih =>
  have hnext := nth_prime_le_prod_add_one n
  have hbound : Nat.nth isPrime n ≤ 2 ^ (2 ^ n) := by
    omega

```

It follows that $P_{n+1} = P_n p_{n+1} \leq P_n B < B^2 = 2^{2^{n+1}}$.

```

rw [Finset.prod_range_succ]
calc
  (∏ i ∈ Finset.range n, Nat.nth isPrime i) * Nat.nth isPrime n ≤
    (∏ i ∈ Finset.range n, Nat.nth isPrime i) * 2 ^ (2 ^ n) :=
    Nat.mul_le_mul_left _ hbound
  < 2 ^ (2 ^ n) * 2 ^ (2 ^ n) :=
    Nat.mul_lt_mul_of_pos_right ih (pow_pos (by omega) _)
  = 2 ^ (2 ^ (n + 1)) := by
    rw [pow_succ (2 : ℕ) n, pow_mul, pow_two]

```

□

Theorem 2.2.9. *For the n th prime p_n , with $n \geq 1$, we have $p_n \leq 2^{2^{n-1}}$.*

```

theorem nth_prime_le_double_exponential (n : ℕ) (hn : 0 < n) :
  Nat.nth isPrime (n - 1) ≤ 2 ^ (2 ^ (n - 1))

```

Proof. Write $n = m + 1$. The preceding lemmas give $p_n \leq P_m + 1$ and $P_m < 2^{2^m}$. Since these are integers, $P_m + 1 \leq 2^{2^m}$, proving the assertion.

```

obtain ⟨m, rfl⟩ := Nat.exists_eq_succ_of_ne_zero (Nat.ne_of_gt hn)
have hproduct := prod_initial_primes_lt m
have hnext := nth_prime_le_prod_add_one m
simp only [Nat.succ_sub_one] using
  (show Nat.nth isPrime m ≤ 2 ^ (2 ^ m) by omega)

```

□

2.3 The Goldbach Conjecture

We examine gaps between primes and restrictions on their distribution. Unless otherwise specified, the variables are natural numbers, and the Lean namespace remains `ElementaryNumberTheory.Chapter02`.

Theorem 2.3.1. *For every positive integer n , there exist n consecutive composite integers.*

```
theorem exists_consecutive_composites (n : ℕ) (_hn : 0 < n) :
  ∃ M : ℕ, ∀ i : ℕ, i < n → isComposite (M + i)
```

Proof. Let $F = (n + 1)!$ and consider $F + 2, F + 3, \dots, F + n + 1$. For $0 \leq i < n$, the integer $i + 2$ divides F , so it also divides $F + i + 2$.

```
let F := (n + 1).factorial
refine (F + 2, ?_)
intro i hi
obtain (t, ht) := factorial_multiple (n + 1) (i + 2) (by omega) (by
  ↪ omega)
have hF : 0 < F := Nat.factorial_pos (n + 1)
have hdiv : i + 2 ∣ F + 2 + i := by
  refine (t + 1, ?_)
  dsimp [F]
  rw [ht]
  ring
```

Each displayed integer exceeds one. Its divisor $i + 2$ is greater than one and is strictly smaller than $F + i + 2$, since $F > 0$. Thus it cannot be prime.

```
refine (by omega, ?_)
intro hp
rcases hp.2 (i + 2) hdiv with hone | heq

• omega
• omega
```

□

Lemma 2.3.2. *A product of two or more integers of the form $4k + 1$, with $k \in \mathbb{Z}$, again has that form. More generally, this holds for any finite list, including the empty product.*

```
theorem prod_four_mul_add_one (l : List ℤ)
  (hl : ∀ a ∈ l, ∃ k : ℤ, a = 4 * k + 1) :
  ∃ k : ℤ, l.prod = 4 * k + 1
```

Proof. Induct on the number of factors. The empty product is $1 = 4 \cdot 0 + 1$. If the first factor is $4u + 1$ and the remaining product is $4v + 1$, then their product is $4(4uv + u + v) + 1$.

```
induction l with
| nil => exact ⟨0, by simp⟩
| cons a l ih =>
  obtain ⟨u, hu⟩ := hl a List.mem_cons_self
  obtain ⟨v, hv⟩ := ih (fun b hb => hl b (List.mem_cons_of_mem a hb))
  refine ⟨4 * u * v + u + v, ?_⟩
  rw [List.prod_cons, hu, hv]
  ring
```

□

The notation $a \bmod 4$ below means the remainder upon division by 4; in Lean it is written `a % 4`. The division algorithm gives $a = 4\lfloor a/4 \rfloor + (a \bmod 4)$ with remainder in $\{0, 1, 2, 3\}$.

Lemma 2.3.3. *A prime p is either 2, or has remainder 1 or 3 upon division by 4.*

```
theorem prime_eq_two_or_mod_four (p : ℕ) (hp : isPrime p) :
  p = 2 ∨ p % 4 = 1 ∨ p % 4 = 3
```

Proof. The remainder lies between zero and three. The desired conclusion already holds when it is one or three.

```
have hmod : p % 4 < 4 := Nat.mod_lt p (by omega)
by_cases hone : p % 4 = 1

· exact Or.inr (Or.inl hone)

· by_cases hthree : p % 4 = 3
  · exact Or.inr (Or.inr hthree)
```

For remainder zero, $p = 2(2\lfloor p/4 \rfloor)$; for remainder two, $p = 2(2\lfloor p/4 \rfloor + 1)$. Thus $2 \mid p$, and primality forces $p = 2$.

```

• have hdiv : 2 | p := by
  rcases (show p % 4 = 0 ∨ p % 4 = 2 by omega) with hzero | htwo

  • exact ⟨2 * (p / 4), by omega⟩
  • exact ⟨2 * (p / 4) + 1, by omega⟩

rcases hp.2 2 hdiv with h | h

• omega
• exact 0r.inl h.symm

```

□

Lemma 2.3.4. *If $n \bmod 4 = 3$, then n has a prime divisor whose remainder upon division by 4 is also 3.*

```

theorem exists_prime_divisor_mod_four_three (n : ℕ) (hn : n % 4 = 3) :
  ∃ p : ℕ, isPrime p ∧ p % 4 = 3

```

Proof. Choose a prime factor list for n and suppose that no prime divisor has remainder three. The factor 2 is impossible, since n has odd remainder. By the preceding lemma every prime factor must therefore have remainder one.

```

classical
obtain ⟨l, hl, hprod⟩ := exists_prime_factors n (by omega)
by_contra h
push Not at h
have hone : ∀ p ∈ l, p % 4 = 1 := by
  intro p hp
  have hdiv : p | n := ⟨(l.erase p).prod, by rw [← hprod,
    ↪ List.prod_erase hp]⟩
  rcases prime_eq_two_or_mod_four p (hl p hp) with htwo | hrest

  • obtain ⟨k, hk⟩ := hdiv
    rw [htwo] at hk
    omega

  • exact hrest.resolve_right (h p (hl p hp) hdiv)

```

For each prime factor p , division by four gives $p = 4\lfloor p/4 \rfloor + 1$. View these equalities as integer equalities to apply the product lemma.

```

have hform :  $\forall a \in l.map \text{ (fun } p : \mathbb{N} \Rightarrow (p : \mathbb{Z}))$ ,  $\exists k : \mathbb{Z}, a = 4 * k + 1$ 
 $\leftarrow$  := by
  intro a ha
  obtain (p, hp, rfl) := List.mem_map.mp ha
  refine ((p / 4 :  $\mathbb{N}$ ), ?_)
  have heq :  $p = 4 * (p / 4) + 1$  := by
    have := hone p hp
    omega
  exact_mod_cast heq

```

The product lemma makes the entire product equal to $4k + 1$ for some integer k . Since that product is n , this contradicts $n \bmod 4 = 3$.

```

obtain (k, hk) := prod_four_mul_add_one _ hform
have hcast : (l.map (fun p :  $\mathbb{N} \Rightarrow (p : \mathbb{Z}))$ ).prod = (n :  $\mathbb{Z}$ ) := by
  rw [ $\leftarrow$  Nat.cast_list_prod, hprod]

rw [hcast] at hk
omega

```

□

Lemma 2.3.5. *For every natural number m , there is a prime $p > m$ of the form $p = 4k + 3$ with $k \in \mathbb{N}$.*

```

theorem exists_prime_mod_four_three_gt (m :  $\mathbb{N}$ ) :
   $\exists p : \mathbb{N}, \text{isPrime } p \wedge m < p \wedge \exists k : \mathbb{N}, p = 4 * k + 3$ 

```

Proof. Put $N = 4m! - 1$. Since $m! \geq 1$, this is positive and has remainder three modulo four. Choose a prime divisor p with that remainder, and write $p = 4\lfloor p/4 \rfloor + 3$.

```

let N := 4 * m.factorial - 1
have hfac := Nat.factorial_pos m
have hN :  $N \% 4 = 3$  := by
  dsimp [N]
  omega
obtain (p, hp, hdiv, hmod) := exists_prime_divisor_mod_four_three N hN
refine (p, hp, ?_, p / 4, by omega)

```

If $p \leq m$, write $m! = pt$. Then $N + 1 = 4m! = p(4t)$, so p divides $N + 1$ as well as N .

```

by_contra h
obtain ⟨t, ht⟩ := factorial_multiple m p (lt_trans Nat.zero_lt_one hp.1)
  ↪ (by omega)
have hsucc : p | N + 1 := by
  refine ⟨4 * t, ?_⟩
  have hNsucc : N + 1 = 4 * m.factorial := by
    dsimp [N]
    omega

  rw [hNsucc, ht]
  ring

```

This contradicts the consecutive-number lemma. Hence $p > m$.

```

exact not_dvd_of_dvd_add_one p N hp hsucc hdiv

```

□

Theorem 2.3.6. *There are infinitely many primes of the form $4k + 3$, with $k \in \mathbb{N}$.*

```

theorem infinite_primes_four_mul_add_three :
  Set.Infinite {p : ℕ | isPrime p ∧ ∃ k : ℕ, p = 4 * k + 3}

```

Proof. The preceding construction produces such a prime above every bound. A finite set of natural numbers is bounded, so this set of primes is infinite.

```

apply Set.infinite_of_forall_exists_gt
intro m
obtain ⟨p, hp, hgt, hform⟩ := exists_prime_mod_four_three_gt m
exact ⟨p, ⟨hp, hform⟩, hgt⟩

```

□

Theorem 2.3.7. *Let $p, d \in \mathbb{N}$ and $n > 2$. If every term of*

$$p, p + d, p + 2d, \dots, p + (n - 1)d$$

is prime, then every prime $q < n$ divides d . The statement includes the constant progression $d = 0$.

```
theorem prime_dvd_common_difference (p d n : ℕ) (hn : 2 < n)
  (hterms : ∀ i : ℕ, i < n → isPrime (p + i * d)) :
  ∀ q : ℕ, isPrime q → q < n → q ∣ d
```

Proof. The first term p is prime. Fix a prime $q < n$. If $d = 0$, then $q \mid d$ immediately.

```
have hp : isPrime p := by
  simp only [zero_mul, add_zero] using hterms 0 (by omega)

intro q hq hqn
by_cases hd : d = 0

· exact {0, by rw [hd, mul_zero]}
```

Suppose $d > 0$. We must have $n \leq p$: otherwise the term with index p occurs in the progression and equals $p + pd = p(1 + d)$. Its divisor $p > 1$ is smaller than the term, contradicting primality.

```
· have hnp : n ≤ p := by
  by_contra h
  have hterm := hterms p (by omega)
  have hdiv : p ∣ p + p * d := {1 + d, by ring}
  rcases hterm.2 p hdiv with hone | heq

· have := hp.1
  omega
· have hdpos : 0 < d := by omega
  nlinarith [hp.1]
```

Assume, seeking a contradiction, that q does not divide d . The gcd of q and d divides the prime q , so it is 1 or q . The latter would imply $q \mid d$; hence the gcd is 1.

```
by_contra hqd
have hqpos : 0 < q := lt_trans Nat.zero_lt_one hq.1
have hqne : (q : ℤ) ≠ 0 := by
  exact_mod_cast (Nat.ne_of_gt hqpos)
obtain (_, hgq, hgd, _, _) :=
  Chapter01.gcd_spec (q : ℤ) (d : ℤ) (0r.inl hqne)
have hg : Chapter01.gcd (q : ℤ) (d : ℤ) = 1 := by
  rcases hq.2 _ (Int.natCast_dvd_natCast.mp hgq) with hone | heq

· exact hone
· rw [heq] at hgd
  exact (hqd (Int.natCast_dvd_natCast.mp hgd)).elim
```

Choose integers r, s with $1 = qr + ds$. Divide $-ps$ by q , obtaining $-ps = aq + i$ with $0 \leq i < q$.

```

obtain (r, s, hrs) :=
  (Chapter01.gcd_eq_one_iff_exists_mul_add_mul (q : ℤ) (d : ℤ) (0r.inl
    ↪ hqne)).mp hg
obtain (⟨a, i⟩, ⟨hrem, hi⟩, _) :=
  Chapter01.existsUnique_quotient_remainder (-(p : ℤ) * s) (q, hqpos)
change -(p : ℤ) * s = a * (q : ℤ) + (i : ℤ) at hrem
change i < q at hi

```

These two equalities give $p + id = q(pr - da)$. Thus $q \mid p + id$.

```

have hdiv : q ∣ p + i * d := by
  apply Int.natCast_dvd_natCast.mp
  refine (⟨(p : ℤ) * r - (d : ℤ) * a, ?_⟩
    push_cast
    nlinarith [congrArg (fun z => (p : ℤ) * z) hrs,
      congrArg (fun z => (d : ℤ) * z) hrem]

```

Since $i < q < n$, this term is prime. Its prime divisor q must equal the term. But $p + id \geq p \geq n > q$, a contradiction. Therefore $q \mid d$.

```

have hterm := hterms i (lt_trans hi hqn)
rcases hterm.2 q hdiv with hone | heq

• have := hq.1
  omega
• omega

```

□

Chapter 3

The Theory of Congruences

3.1 Basic Properties of Congruence

3.2 Binary and Decimal Representations of Integers

3.3 Linear Congruences and the Chinese Remainder Theorem

Chapter 4

Fermat's Theorem

4.1 Fermat's Little Theorem and Pseudoprimes

4.2 Wilson's Theorem

4.3 The Fermat-Kraitchik Factorization Method

Chapter 5

Number-Theoretic Functions

5.1 The Sum and Number of Divisors

5.2 The Möbius Inversion Formula

5.3 The Greatest Integer Function

5.4 An Application to the Calendar

Chapter 6

Euler's Generalization of Fermat's Theorem

6.1 Euler's Phi-Function

6.2 Euler's Theorem

6.3 Some Properties of the Phi-Function

Chapter 7

Primitive Roots and Indices

7.1 The Order of an Integer Modulo n

7.2 Primitive Roots for Primes

7.3 Composite Numbers Having Primitive Roots

7.4 The Theory of Indices

Chapter 8

The Quadratic Reciprocity Law

8.1 Euler's Criterion

8.2 The Legendre Symbol and Its Properties

8.3 Quadratic Reciprocity

8.4 Quadratic Congruences with Composite Moduli

Chapter 9

Introduction to Cryptography

9.1 From Caesar Cipher to Public Key Cryptography

9.2 The Knapsack Cryptosystem

9.3 An Application of Primitive Roots to Cryptography

Chapter 10

Numbers of Special Form

10.1 Perfect Numbers

10.2 Mersenne Primes and Amicable Numbers

10.3 Fermat Numbers

Chapter 11

Certain Nonlinear Diophantine Equations

11.1 The Equation $x^2 + y^2 = z^2$

11.2 Fermat's Last Theorem

Chapter 12

Representation of Integers as Sums of Squares

12.1 Sums of Two Squares

12.2 Sums of More Than Two Squares

Chapter 13

Fibonacci Numbers

13.1 The Fibonacci Sequence

13.2 Certain Identities Involving Fibonacci Numbers

Chapter 14

Continued Fractions

14.1 Finite Continued Fractions

14.2 Infinite Continued Fractions

14.3 Farey Fractions

14.4 Pell's Equation

Chapter 15

Some Modern Developments

15.1 Primality Testing and Factorization

15.2 An Application to Factoring: Remote Coin Flipping

15.3 The Prime Number Theorem and Zeta Function

Bibliography

- [1] David M. Burton, *Elementary Number Theory*, McGraw-Hill International Editions.